1. Importazione della libreria

python

import requests

- import serve a importare una libreria.
- requests è una libreria Python che permette di inviare richieste HTTP e ricevere risposte dai server web.

In questo programma viene utilizzata per provare diversi metodi HTTP, come GET, POST e DELETE.

Se la libreria non è installata, puoi installarla con:

bash

pip install requests

2. Inserimento dell'URL target

python

url = input("Inserisci l'URL target: ")

Questa istruzione chiede all'utente di inserire l'indirizzo del server da analizzare.

- input() mostra il messaggio e attende che l'utente scriva qualcosa.
- url è la variabile in cui viene memorizzato il valore inserito.

Per esempio, puoi inserire:

text

https://example.com/

Da quel momento, la variabile url conterrà quell'indirizzo e verrà utilizzata nelle richieste successive.

Nota: il programma non controlla se l'URL inserito è valido prima di iniziare i test.

3. Elenco dei metodi HTTP

python

metodi = ["GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH", "HEAD", "TRACE"]

Questa riga crea una lista chiamata metodi, contenente gli otto metodi HTTP da testare.

Metodo	**Funzione principale**
GET	Richiede una risorsa al server.

POST	Invia dati al server, per esempio per creare una risorsa o inviare un modulo.
PUT	Crea o sostituisce una risorsa, a seconda dell'implementazione.
DELETE	Richiede l'eliminazione di una risorsa.
OPTIONS	Richiede informazioni sulle opzioni di comunicazione supportate.
PATCH	Richiede una modifica parziale di una risorsa.
HEAD	Richiede gli header della risposta senza il corpo.
TRACE	Richiede al server di restituire informazioni sulla richiesta ricevuta; può essere disabilitato per motivi di sicurezza.

La lista serve a evitare di scrivere otto volte lo stesso blocco di codice.

4. Ciclo per provare ogni metodo

python

for metodo in metodi:

Questa istruzione avvia un ciclo for.

- for significa che il programma ripete delle operazioni.
- metodo è la variabile che, a ogni iterazione, riceve un elemento della lista.
- in metodi indica che gli elementi vengono presi dalla lista metodi.

Il ciclo eseguirà quindi otto iterazioni:

1. GET
2. POST
3. PUT
4. DELETE
5. OPTIONS
6. PATCH
7. HEAD
8. TRACE

A ogni iterazione, il programma invia una richiesta utilizzando il metodo corrente.

5. Gestione delle possibili eccezioni

python

try:

La parola chiave try apre un blocco di codice nel quale potrebbero verificarsi errori.

Per esempio, una richiesta potrebbe fallire perché:

- Il server non è raggiungibile.
- La connessione scade.
- Il certificato TLS non è valido.
- Si verifica un problema di rete.

Se si verifica un'eccezione del tipo gestito dal successivo except, il programma può mostrare un messaggio d'errore senza interrompere necessariamente l'intero ciclo.

6. Invio della richiesta HTTP

python

```
risposta = requests.request(
    metodo,
    url,
    timeout=5,
    allow_redirects=False
)
```

Questa è la parte centrale del programma: invia effettivamente la richiesta HTTP.

Vediamo i parametri.

- requests.request(...): invia una richiesta HTTP.
- metodo: indica il metodo utilizzato in quel momento, per esempio GET oppure DELETE.
- url: specifica l'indirizzo del server.
- timeout=5: imposta un limite di attesa di cinque secondi per l'operazione di rete.
- allow_redirects=False: impedisce a Requests di seguire automaticamente i reindirizzamenti HTTP, come quelli indicati dagli status code 301, 302 o 307.

La risposta ricevuta viene memorizzata nella variabile risposta.

Per esempio, il server potrebbe restituire:

text

Status code: 200

oppure:

text

Status code: 405

Il codice di stato sarà disponibile attraverso risposta.status_code.

7. Controllo del codice di stato

python

if risposta.status_code not in [405, 501]:

Questa istruzione controlla il codice HTTP restituito dal server.

- risposta.status_code contiene il codice di stato della risposta.
- not in significa “non è presente in”.
- [405, 501] è la lista dei due codici considerati dal programma come indicativi di un metodo non consentito o non supportato.

In particolare:

- 405 Method Not Allowed: il server non consente quel metodo per la risorsa richiesta.
- 501 Not Implemented: il server non supporta la funzionalità richiesta.

Se il codice ricevuto è diverso da 405 e 501, il programma entra nel blocco if.

Attenzione: questa condizione non dimostra che il metodo sia realmente abilitato. È soltanto una classificazione semplificata basata sul codice di stato.

8. Stampa del risultato positivo

python

print(f"{metodo}: potenzialmente abilitato ({risposta.status_code})")

Se la condizione precedente è vera, il programma stampa il metodo e il codice HTTP ricevuto.

La f prima della stringa indica una *f-string*, che permette di inserire variabili nel testo tramite parentesi graffe.

Per esempio, se il metodo è GET e il server risponde con 200, vedrai:

text

GET: potenzialmente abilitato (200)

La parola **potenzialmente** è importante: il codice non verifica se l'operazione sia effettivamente autorizzata o se possa essere completata con successo.

9. Caso alternativo: metodo non supportato

python

else:

    print(f"{metodo}: non supportato ({risposta.status_code})")

else viene eseguito quando la condizione dell'if è falsa, cioè quando il codice di stato è 405 oppure 501.

Per esempio:

text

DELETE: non supportato (405)

Il programma indica quindi che il server ha risposto con un codice comunemente associato a un metodo non consentito o non implementato.

Questo non significa necessariamente che il metodo sia sempre vietato: il comportamento può cambiare in base al percorso, alla configurazione del server o alle condizioni della richiesta.

10. Gestione degli errori della richiesta

python

except requests.RequestException:

Questa istruzione intercetta le eccezioni derivate da requests.RequestException, cioè diversi errori che possono verificarsi durante l'invio della richiesta HTTP.

Esempi:

- Timeout della richiesta.
- Errori di connessione.
- Errori relativi al certificato TLS.

La riga successiva:

python

print(f"{metodo}: errore nella richiesta")

stampa un messaggio indicando quale metodo ha generato l'errore.

Per esempio:

text

PATCH: errore nella richiesta

Il programma poi passa al metodo successivo del ciclo, se l'errore è stato gestito normalmente.

Il messaggio non specifica la causa dell'errore, perché l'eccezione non viene assegnata a una variabile né stampata.

Riassunto del funzionamento

1. L'utente inserisce l'URL

2. Il programma legge la lista degli otto metodi HTTP

3. Invia una richiesta per ogni metodo

4. Analizza la risposta

- Status diverso da 405 e 501 → potenzialmente abilitato.
- Status 405 o 501 → non supportato.
- Eccezione di rete → errore nella richiesta.

5. Stampa il risultato e passa al metodo successivo